

# AI Usage Note: Helpdesk Ticket Management MCP System

This document outlines the collaborative role of Artificial Intelligence (AI) and human engineering in building, debugging, and documenting the Helpdesk Ticket Management Model Context Protocol (MCP) System.

## 1. AI Tools Used

|                |                                                                                |
|----------------|--------------------------------------------------------------------------------|
| Antigravity    | Primary assistant for code generation, workspace navigation, and verification. |
| Claude Desktop | Runtime environment for hosting & testing FastMCP server stdio tools.          |
| ChatGPT        | Assisted with initial patterns, architecture planning, and boilerplate.        |
| Gemini API     | Programmatic ticket classification, priority labeling, and summarization.      |

## 2. What AI Helped With

- MCP Server Tools:** Registered database stubs: list, get, create, update, comment, search\_kb.
- ReAct Agent Loop:** Guided stateful reasoning, validations, and ticket-matching checks.
- Documentation:** Drafted architecture files, setup procedures, and integration guides.
- Boilerplate Code:** Generated FastAPI controllers, Pydantic schemas, and SQLAlchemy models.

## 3. What AI Got Wrong

- Placeholder Paths:** Hardcoded absolute directory paths and incorrect local imports.
- Missing Imports:** Omitted typing extensions and introduced circular schema imports.
- Transaction Issues:** Wrote unsafe SQLite operations leading to constraint/duplicate insertion failures.
- Security Configuration:** Lacked production CORS settings, secrets management, and custom thresholds.

## 6. Best Prompts Used Registry

A. MCP Architecture

**Purpose:** Expose SQLite tools via FastMCP to Claude Desktop.

"Write a stdio-based MCP server using the FastMCP Python SDK that exposes a SQLite database for helpdesk ticket management. The server should register tools to read, write, update, and search tickets and comments. Provide a JSON configuration file template that can be loaded into Claude Desktop to interact with the server locally."

B. Agent Workflow

**Purpose:** Stateful reasoning, duplicate detection & routing.

"Design a stateful ReAct (Reasoning + Acting) loop for a support agent in Python. The agent must first classify a ticket, query a knowledge base for answers, search the active database for duplicate tickets using a similarity threshold, and then decide whether to comment on a duplicate or create a new ticket, explaining its thoughts at each step."

C. Gemini Classifier

**Purpose:** Structured response schema with API fallbacks.

"Create a Python service gemini\_classifier.py that interfaces with the Google Gemini API (specifically gemini-2.5-flash). Force the output to be a structured JSON object containing keys for priority, category, and summary by configuring response\_mime\_type='application/json'. Implement a fallback to local keyword classification rules and default values if the API key is missing or the request fails."

- Code Hardening:** Reviewed AI scripts and resolved path, package, and import failures.
- DB Fixes:** Patched duplicate inserts, added constraints, and safe SQLite transactions.
- Tool Verification:** Verified FastMCP tool stdio execution and schema responses.
- Integrations:** Configured Claude Desktop json configuration and resolved local setup.
- Security Controls:** Managed CORS origins, environment secrets, and JWT expiration tokens.
- Similarity Rules:** Calibrated duplication matching thresholds (Jaccard similarity).
- Gemini Validation:** Ensured Gemini structured JSON parsing and implemented robust logic fallbacks.
- Business Logic:** Wrote custom ticketing logic and verified search KB integration.

## 4. Human Contributions (Hardening)

## 5. Collaborative Conclusion

The final Helpdesk Ticket Management MCP System represents a collaborative integration of rapid AI code generation and disciplined human validation. AI accelerated initial prototyping, structural scaffolding, and documentation templates. However, human oversight, debugging, transaction correction, and security alignment were indispensable to transform draft components into a secure, reliable, and production-ready tool integration ecosystem.